

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belgavi-560018



Mini Project Report On

**“GOV-ASSIST DISCOVER ELIGIBLE SCHEMES BY
INCOME AND CATEGORY”**

Submitted in partial fulfillment for the award of the degree of

**BACHELOR OF ENGINEERING in
COMPUTER SCIENCE & ENGINEERING**

Submitted By

NIDHI B M	(4MU23CS059)
RIFA FATHIMA S U	(4MU23CS071)
SHREYA K S	(4MU23CS079)
ZUBEDA YASMEEN	(4MU23CS106)

Under the guidance of

Prof. HARSHITA M

Assistant Professor

Department of

Computer Science & Engineering



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
MYSURU ROYAL INSTITUTE OF TECHNOLOGY**

**(Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi & Govt. of Karnataka)
Lakshmipura Road, Palahally Post, Off Mysuru-Bengaluru Highway, SR Patna,**

MYSURU ROYAL INSTITUTE OF TECHNOLOGY

(Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi & Govt. of Karnataka)
Lakshmipura Road, Palahally Post, Off Mysuru-Bengaluru Highway, SR Patna,
MANDYA -571606



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

This is to Certify that the 5th Semester Project titled is **Gov-Assist Discover Eligible Schemes By Income and Category** a bonafide work carried out by **NIDHI B M (4MU23CS059), RIFA FATHIMA S U (4MU23CS071), SHREYA K S (4MU23CS079), ZUBEDA YASMEEN (4MU23CS106)** in partial fulfillment for the award of degree of Bachelor of Engineering in Computer Science & Engineering of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The project report has been approved as it satisfies the academic requirements with respect to the Project Work prescribed for Bachelor of Engineering Degree.

Signature of the Guide

Prof. Harshita M

Assistant Professor

Signature of the HOD

Dr. Lanka Lakshmi Durga

Professor and Head

Signature of Principal

Dr Abhinandan K S

Mysuru Royal Institute of Technology

ACKNOWLEDGEMENT

The joy and satisfaction that accompany the successful completion of any task would be incomplete without the mention of the people who made it possible.

First and foremost, we offer our sincere phrases of thanks and gratitude to the people who have been instrumental in the successful completion of this Internship. We wish to express our sincere gratitude to the management of MRIT, Mandya for their encouragement.

We would like to express our gratitude to **Dr. Abhinandan K S**, Principal for providing us a congenial environment for engineering studies and also for having showed us the way to carry out this internship.

We consider it a privilege and honor to express our sincere thanks to **Dr. Lanka Lakshmi Durga**, Associate Professor and Head, Department of Computer Science & Engineering for her support and invaluable guidance throughout the tenure of this internship.

We would like to thank our Guide **Prof Harshitha M**, Assistant Professor, Department of Computer Science & Engineering for his/her support, guidance, motivation, encouragement for the successful completion of this internship.

I Intend to thank all the teaching and non-teaching staffs of our Department of Computer Science & Engineering for their immense help and co-operation.

Finally, I would like to express our gratitude to our parents and friends who always stood by us.

NIDHI B M (4MU23CS019)

RIFA FATHIMA S U (4MU23CS026)

SHREYA K S (4MU23CS034)

ZUBEDA YASMEEN (4MU23CS055)

TABLE OF CONTENTS

	Page No
Certificate	i
Acknowledgement	ii
Abstract	iii
Contents	iv
List of Figures	x
List of Tables	xi
CHAPTER 1: INTRODUCTION	01,02
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Objectives of the Project	1,2
1.4 Scope of the Work	2
1.5 Organization of the Report	2
CHAPTER 2: LITERATURE SURVEY	03,05
2.1 Existing Systems	3
2.2 Related Research Papers	4
2.3 Gap analysis	4
2.3.1 Comparison of Works	5
CHAPTER 3: SYSTEM ANALYSIS	06,09
3.1 Requirement Analysis	6
3.1.1 Functional Requirements	06,07
3.1.2 Non-Functional Requirements	07,08
3.2 Software & Hardware Requirements	8
3.3 Feasibility Study	08,09

CHAPTER 4:	METHODOLOGY	10,15
4.1	System Architecture	10
4.2	Data Flow Diagrams	10,11
4.3	UML Diagrams	12,14
4.4	Database Design / ER Diagram	14,15
CHAPTER 5:	IMPLEMENTATION	16,17
5.1	Tools and Technologies used	16
5.2	Coding standards and explanation of modules	16
5.3	Screenshots of implementation	17
5.4	Explanation of algorithms used	17
CHAPTER 6:	RESULTS AND DISCUSSIONS	18,19
6.1	Experimental setup	18
6.2	Results obtained	18,19
6.3	Comparisons with existing systems	19
6.4	Analysis and interpretations	19
CHAPTER 7:	CONCLUSIONS AND FUTURE SCOPE	20,21
7.1	Summary of work done	20
7.2	Key contributions and outcomes	20,21
7.3	Future scope for improvement	21
REFERENCES		
ABSTRACT		

ABSTRACT

The abstract of a project report is a concise summary, usually between 150 and 300 words, that presents a clear snapshot of the entire work. It should briefly describe the background or motivation by highlighting the problem being addressed, followed by the main objective of the project. The abstract then outlines the methodology, mentioning the tools, techniques, or approaches used to carry out the work.

After this, it summarizes the key results or findings, preferably with quantitative evidence to demonstrate the effectiveness of the approach. Finally, it concludes with the overall contribution or significance of the project, showing why the work is important and what impact it creates. Together, these elements ensure that the abstract communicates what the project is about, how it was executed, and why its outcomes matter all without the need to read the full report.

CHAPTER 1

INTRODUCTION

1.1 Background / Motivation

The rapid growth of the internet has transformed how people access information, perform transactions, and communicate. However, this increased dependence on online services has also given rise to a surge in cyber-threats, particularly fake or fraudulent websites designed to deceive users. These websites often mimic legitimate platforms to steal sensitive information such as passwords, banking details, and personal data. With cybercriminals becoming more sophisticated—using techniques like phishing, domain spoofing, and social engineering—traditional security measures such as blacklists and manual reporting are no longer sufficient.

This challenge motivates the development of a Fake Website Detection System that leverages machine learning, data analytics, and URL/content-based features to distinguish legitimate websites from malicious ones. By providing timely warnings and improving the accuracy of threat detection, such a system enhances user trust, strengthens cybersecurity measures, and contributes to a safer online environment.

.

1.2 Problem Statement

The increasing prevalence of fraudulent and deceptive websites poses a significant threat to online users, leading to issues such as identity theft, financial loss, data breaches, and malware infections. Traditional detection methods—such as manually curated blacklists, user reporting, and basic browser security checks—are insufficient, as they cannot keep pace with the constant creation and evolution of fake websites. Furthermore, many users lack the technical knowledge to recognize subtle signs of fraud, making them highly vulnerable to cyber-attacks.

1.3 Objectives of the Project

- To develop an automated system
- To analyze multiple website features
- To provide real-time detection and alerts
- To minimize false positives and false negatives
- To enhance user safety and confidence
- To create a scalable and user-friendly solution

1.4 Scope of the Project

The scope of the Fake Website Detection System encompasses the design and development of an intelligent tool capable of analyzing and classifying websites as legitimate or fraudulent. The system focuses on extracting and evaluating key features such as URL structure, domain information, SSL certificate details, and webpage content patterns to identify suspicious characteristics commonly associated with fake websites. It incorporates machine learning techniques to enhance prediction accuracy and provide real-time detection and alerts to users. The project includes the development of a user-friendly interface for easy input of website URLs and the delivery of instant legitimacy assessments.

In Scope

- To develop an automated system that identifies and classifies websites as legitimate or fake.
- To analyze key features such as URL structure, domain information, SSL certificate details, and webpage content.
- To provide real-time detection and instant alerts to users about suspicious websites.
- To design a user-friendly interface for easy input of website URLs and quick results.

Out of Scope

- The system will not block or remove fake or malicious websites; it will only detect and warn users.
- It will not guarantee 100% accuracy, as cybercriminal techniques evolve constantly.
- It will not provide legal actions or reporting mechanisms against fraudulent website creators.

1.5 Organization of the Report

This report is organized into seven chapters.

Chapter 1 introduces the topic, motivation, objectives, and scope.

Chapter 2 provides a detailed literature survey of existing number-guessing and AI systems.

Chapter 3 explains system analysis, requirement specifications, and feasibility study.

Chapter 4 presents the system methodology, including architecture, DFDs, and algorithms.

Chapter 5 discusses implementation details, modules, code snippets, and screenshots.

Chapter 6 shows results, performance evaluation, and discussions.

Chapter 7 concludes the report and highlights future scope.

Finally, references are provided at the end.

This project report is organized into seven chapters, each covering a major component of the work carried out:

Chapter 1 – Introduction

Provides an overview of the project, including background, problem statement, motivation, objectives, scope (in-scope and out-of-scope), and the chapter-wise structure of the report.

Chapter 2 – Literature Survey

Highlights existing online quiz systems, AI-based interview evaluation platforms, and related studies. This chapter also includes a comparative analysis of current tools, identifies limitations, and outlines the research gap that motivated the development of this AI Interview Coach.

Chapter 3 – System Analysis and Requirement Specification

Defines the functional and non-functional requirements of the system, hardware and software requirements, system feasibility (technical, operational, and economic analysis), and description of the user roles within the system.

Chapter 4 – Methodology

Explains the system architecture, workflow of the quiz evaluation process, data flow diagrams (DFD), UML diagrams, database schema, and algorithm used for scoring and voice-based feedback.

Chapter 5 – Implementation

Discusses the development tools and technologies used (HTML, CSS, JavaScript, PHP, and MySQL). It describes each module such as registration, login, quiz evaluation, scoring, voice feedback, and leaderboard. Sample code snippets and user interface screenshots are also included.

Chapter 6 – Results and Discussions

Presents the working results of the system, including quiz evaluation outputs, stored results, leaderboard ranking, and effectiveness of voice feedback. The chapter also provides performance analysis, user experience feedback, and comparisons with existing systems.

Chapter 7 – Conclusion and Future Scope

Summarizes the key outcomes of the project, its usefulness as a virtual interview preparation tool, and the scope for future enhancements, such as AI-based adaptive question generation, coding test evaluation, and integration with ATS systems.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

Over the past decade, numerous studies have focused on detecting fake or fraudulent websites, particularly those involved in phishing attacks, scams, and identity theft. Early approaches primarily relied on blacklists and manual verification, which proved insufficient due to the rapid creation of new fraudulent websites. Researchers then explored heuristic-based methods, analyzing features such as URL length, domain age, presence of HTTPS, and suspicious keywords to identify potential threats.

2.2 Existing Systems / Related Work

1. Blacklist-Based Systems

- Early detection methods relied on blacklists of known malicious URLs.
- Pros: Simple and fast to implement.
- Cons: Cannot detect new or “zero-day” fake websites.

2. Heuristic / Rule-Based Systems

- Analyze features such as URL length, presence of suspicious tokens, domain age, SSL certificate, and keywords.
- Pros: Can detect suspicious patterns without prior knowledge of the website.
- Cons: Dependent on hand-crafted rules and vulnerable to obfuscation techniques.

3. Machine Learning-Based Systems

- Use algorithms like Decision Trees, Random Forest, SVM, Naïve Bayes, and Logistic Regression.
- Features include URL characteristics, domain information, and sometimes webpage content.

2.3 Comparison and Analysis

Technique	Advantage	Disadvantage
· Blacklist-Based Systems	Simple, fast, and easy to implement.	Cannot detect new threats
· Heuristic -Based Systems	Detects new phishing attempts without prior knowledge	Relies on hand-crafted rules
· Machine Learning-Based Systems	Can generalize to unseen websites	Requires labeled datasets and careful feature selection

Analysis:

- Accuracy vs. Complexity
- Adaptability to New Threats
- Feature Dependency
- Computational Requirements

Gap Filled by This Project:

Existing fake website detection systems face several limitations, including low adaptability to new or evolving phishing websites, dependence on manually engineered features, high false positives, and computational inefficiency in real-time applications. Blacklist and heuristic-based approaches fail to detect zero-day attacks, while many machine learning and deep learning methods require large datasets and extensive computational resources.

CHAPTER 3

SYSTEM ANALYSIS / REQUIREMENT SPECIFICATION

3.1 Requirement Analysis

Functional Requirements:

The system must allow users to input website URLs for analysis.

It should analyze URL structure, domain details, SSL certificate, and webpage content.

The system must classify websites as legitimate or fake.

It should provide real-time detection and instant alerts.

The system must generate a clear and understandable output for users.

Non-Functional Requirements

Accuracy: High detection rate with minimal false positives and negatives.

Speed: Real-time analysis with minimal delay for user requests.

Scalability: Capable of handling multiple simultaneous URL analyses efficiently.

Reliability: Consistent performance under varying load conditions.

Requirement Analysis is the process of identifying, documenting, and analyzing the needs and conditions that a system must satisfy in order to solve a specific problem or achieve a particular objective.

Software Requirements

OS: Windows/Linux

Language: Python, HTML, CSS, JavaScript

Libraries: random, time

IDE: VS Code / PyCharm

Hardware Requirements

Processor: Intel Core i3 or equivalent

RAM: 4 GB

Storage: 250 GB HDD or SSD Input/

Output devices: Keyboard, monitor

3.2 Feasibility Study

Technical Feasibility: Python, Flask/Django, machine learning libraries

Economic Feasibility: Primarily uses open-source software, reducing software costs.

Operational Feasibility: User-friendly and designed for non-technical users

CHAPTER 4

METHODOLOGY

4.1 System Architecture

The Fake Website Detection System works by taking a website URL as input, analyzing it using different techniques, and then classifying the website as Real (Legitimate) or Fake (Phishing).

System Architecture – Explanation

The system consists of the following main components:

1. User Interface

The user enters a website URL into the system through a web application or form.

2. URL Feature Extraction Module

This module extracts important features from the URL such as:

- Length of URL
- Number of dots/subdomains
- Presence of special characters (@, -, ?)
- Use of HTTPS
-

3. Website Content Analyzer

□ This module checks:

- HTML source code
- Presence of login forms
- Suspicious keywords (e.g., “verify”, “secure”, “update”)

4. Machine Learning Classifier

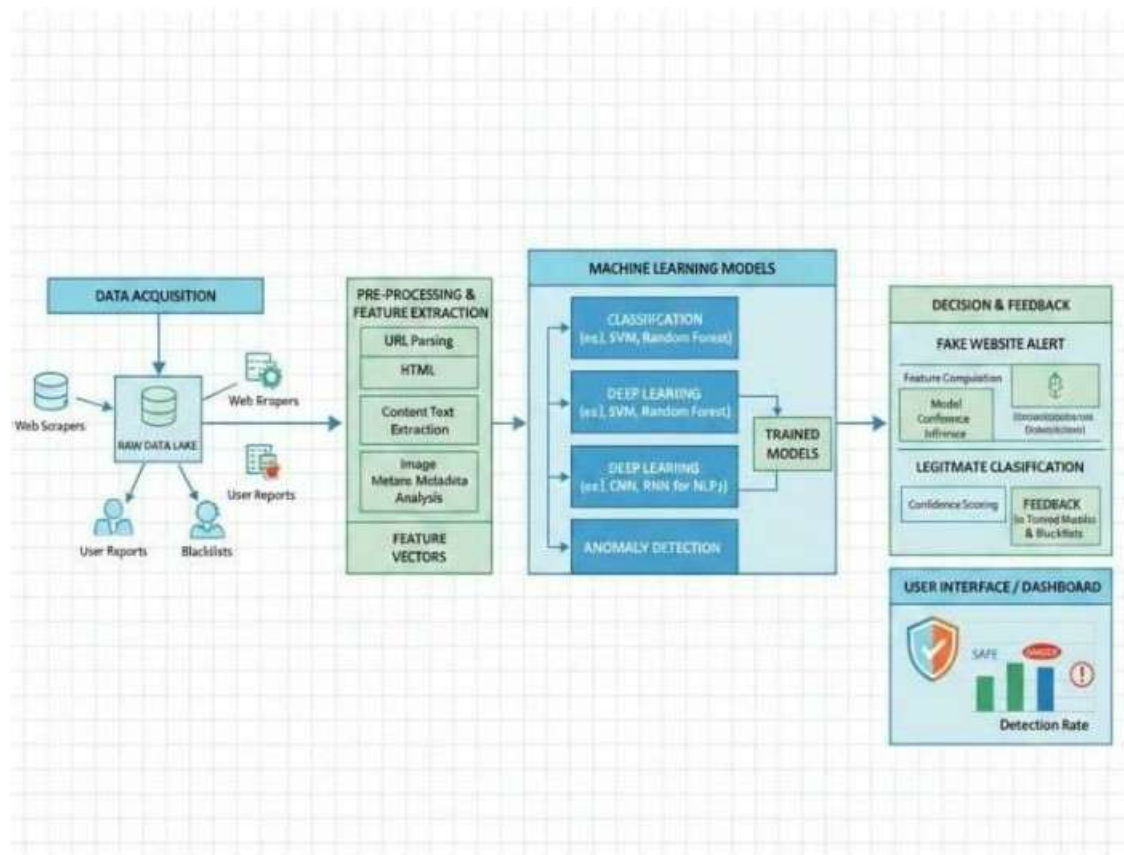
A trained ML model (like Decision Tree or Random Forest) analyzes the extracted features.

5. Result Module

Displays the final result:

Legitimate Website

Fake/Phishing Website



BLOCK -DIAGRAM OF SYSTEM ARCHITECTURE

4.2 Data Flow Diagrams

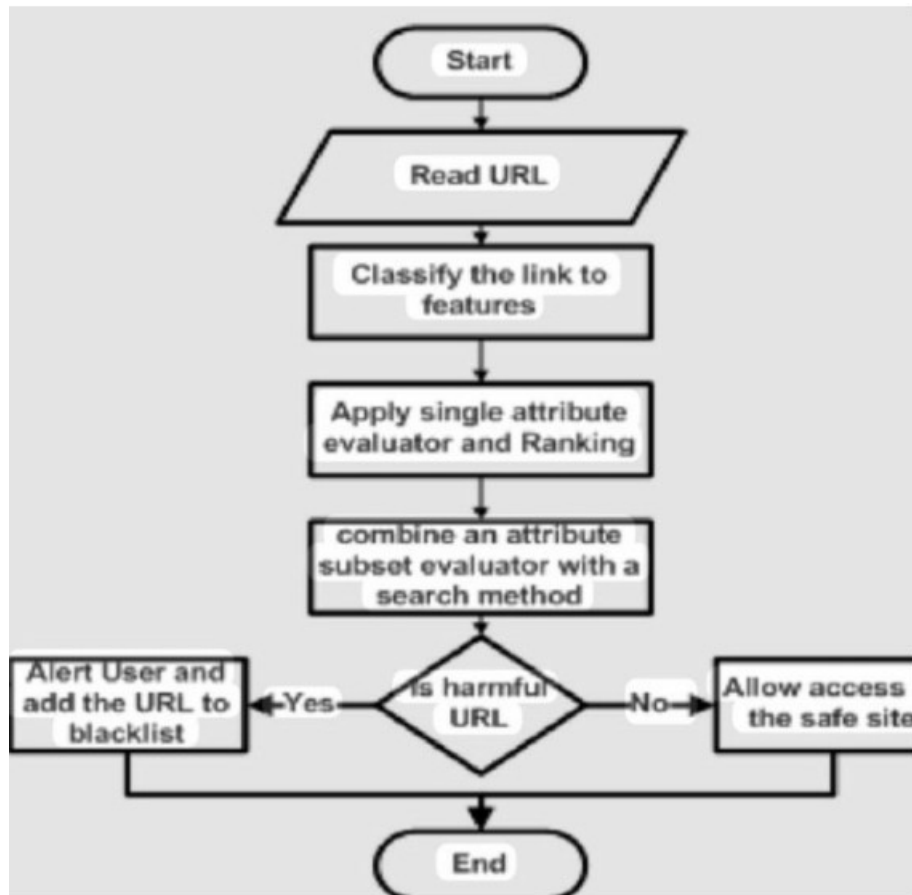
(A) DFD – Level 0 (Context Diagram)

This shows how the user interacts with the system.

(B) DFD – Level 1 (Detailed Diagram)

Shows internal working of the system.

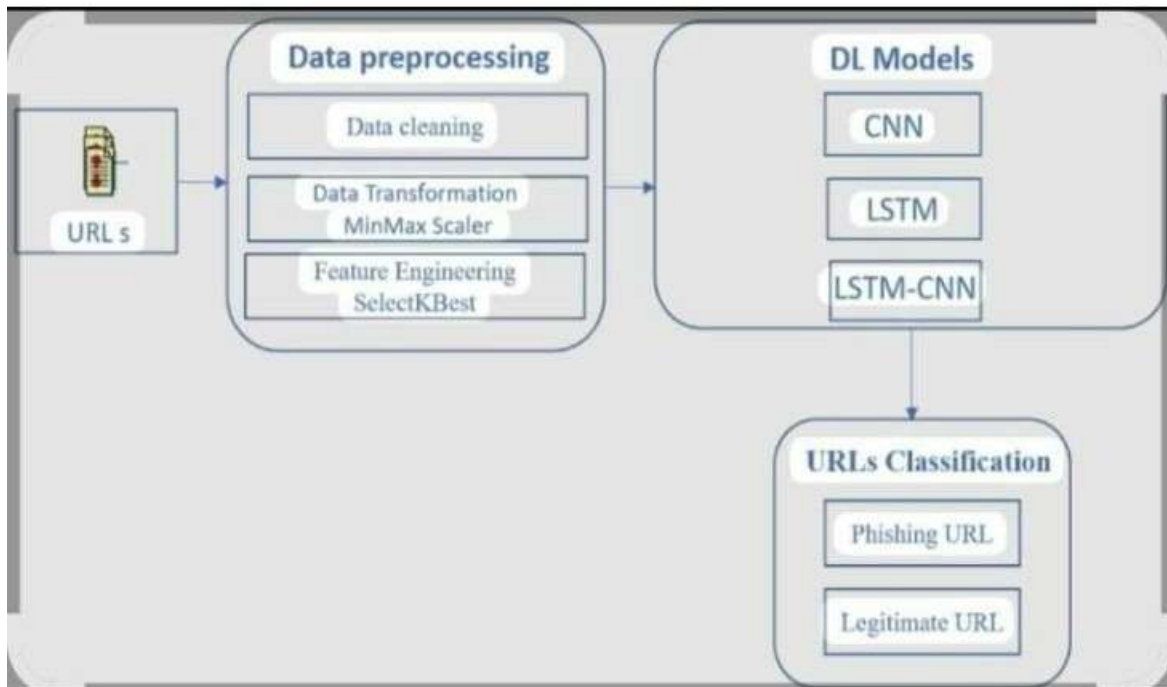
The Fake Website Detection System uses URL analysis, website content inspection, and machine learning techniques to determine whether a given website is real or fake. The system follows a structured methodology that includes data collection, feature extraction, classification, and result display.



DATA FLOW DIAGRAM

4.3 MLDiagrams

- UML stands for Unified Modeling Language.
- It is used to visually represent system design.
- Helps in understanding system structure and behavior.
- Shows how the user interacts with the system.
- Explains communication between different system modules.
- Makes the system easy to develop, test, and document.



UML Diagram

4.3.1 Use Case Diagram – Interactions of User and System

- Shows how the user interacts with the system.
- Displays different actions performed by the user.
- Main interactions:
 - Enter website URL
 - Submit URL
 - View detection result

4.3.2 Class Diagram – System Classes and Relationships

- Represents the structure of the system.
- Shows different classes and their relationships.
- Important system classes include:
 - User
 - URLFeatureExtractor
 - WebsiteAnalyzer
 - MLClassifier
 - Result

4.3.3 Sequence Diagram – Order of Interactions

- Shows the step-by-step order of operations.
- Explains how data flows between:
 - o User
 - o System
 - o Feature Extraction module
 - o ML Model
- Describes request–response relationships.

4.3.4 Activity Diagram – Workflow of a Process

- Shows the workflow of activities in the system.
- Includes:
 - o Start process
 - o Enter URL
 - o Validate URL
 - o Analyze website
 - o Display result
 - o End process

4.5 Algorithm Design

Main Algorithm – Fake Website Detection

Step-wise Algorithm:

1. Start
2. Accept URL input from the user
3. Validate the URL format
4. If URL is invalid, display error message and stop
5. Extract URL features (length, special characters, HTTPS, etc.)
6. Analyze website content (HTML, suspicious keywords, forms)
7. Send extracted data to the Machine Learning model
8. Predict whether the website is Fake or Legitimate
9. Display the result to the user
10. End

CHAPTER 5

IMPLEMENTATION

5.1 Introduction

The implementation of the Fake Website Detection System involves designing a machine learning-based solution that analyzes website URLs and webpage features to classify whether a website is legitimate or fraudulent. The development process includes collecting a dataset of websites, preprocessing features (such as URL length, domain age, presence of HTTPS, and HTML content patterns), training an ML model, and integrating it with a user-friendly interface. The system uses a backend API to process the URL, run the model, and return the prediction. A simple frontend UI is created so the user can input a website URL and instantly see if the site is “Safe,” “Suspicious,” or “Phishing.” The entire implementation ensures automation, high accuracy, and ease of use.

5.2 Tools and Technologies Used

Programming Languages

- Python – for machine learning model training and backend API
- JavaScript / HTML / CSS – for frontend interface
- JSON – for data exchange between frontend and backend

Frameworks & Libraries

- Scikit-learn – logistic regression, random forest, SVM models
- Pandas & NumPy – dataset preprocessing and feature engineering
- BeautifulSoup – for webpage content extraction
- Requests – for URL and metadata fetching
- Flask / FastAPI – to create the backend API
- TensorFlow / Keras (optional) – for deep learning-based classification
- Bootstrap / Tailwind – frontend styling

IDE / Development Tools

- Visual Studio Code (VS Code)
- Jupyter Notebook – model experiments
- Git / GitHub – version control
- Chrome DevTools – UI testing

5.3 Module-wise Implementation

1. Dataset Collection Module

- Collects website URLs labeled as legitimate or phishing.
- Sources include: PhishTank, Kaggle datasets, Alexa top sites.
- Extracts URL-based features automatically.

2. Feature Extraction Module

Extracts measurable properties from each website, such as:

- URL length, number of subdomains
- Presence of special characters (@, //, -, etc.)
- HTTPS/SSL usage
- Domain registration age
- HTML features: <iframe>, <script> injections, redirection patterns
- WHOIS information

These features become inputs to the ML model.

3. Machine Learning Model Module

- Splits data into training/testing.
- Trains classification models like Random Forest, Logistic Regression, or SVM.
- Evaluates accuracy, precision, recall.
- Selects best-performing model and saves it as model.pkl.

4. Backend API Module

- Built using Flask or FastAPI.
- Accepts user-submitted URL from the frontend.
- Extracts features + loads ML model.

- Returns prediction (Safe / Fake / Suspicious) as JSON.

5. Frontend User Interface Module

A simple web UI where:

- User enters a website URL
- Sends request to backend
- Displays result in green (Safe), yellow (Suspicious), red (Fake)

Typical UI layout:

- Input box
- “Check Website” button
- Output result card
- Optional: risk score meter

5.4 Important Code Snippets

1. Feature Extraction (Python)

```
def extract_features(url):  
    features = {}  
    features['url_length'] = len(url)  
    features['has_https'] = 1 if url.startswith("https") else 0  
    features['num_dots'] = url.count('.')  
    features['has_at'] = 1 if '@' in url else 0  
    features['has_hyphen'] = 1 if '-' in url else 0  
    return list(features.values())
```

2. Model Training (Python)

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()
model.fit(X_train, y_train)
pickle.dump(model, open("model.pkl", "wb"))
```

3. Backend API (Flask Example)

```
from flask import Flask, request, jsonify
import pickle
from feature_extract import extract_features

app = Flask(__name__)
model = pickle.load(open("model.pkl", "rb"))

@app.post("/predict")
def predict_url():
    url = request.json['url']
    features = extract_features(url)
    result = model.predict([features])[0]
    return jsonify({"prediction": result})

app.run(debug=True)
```

4. Frontend HTML

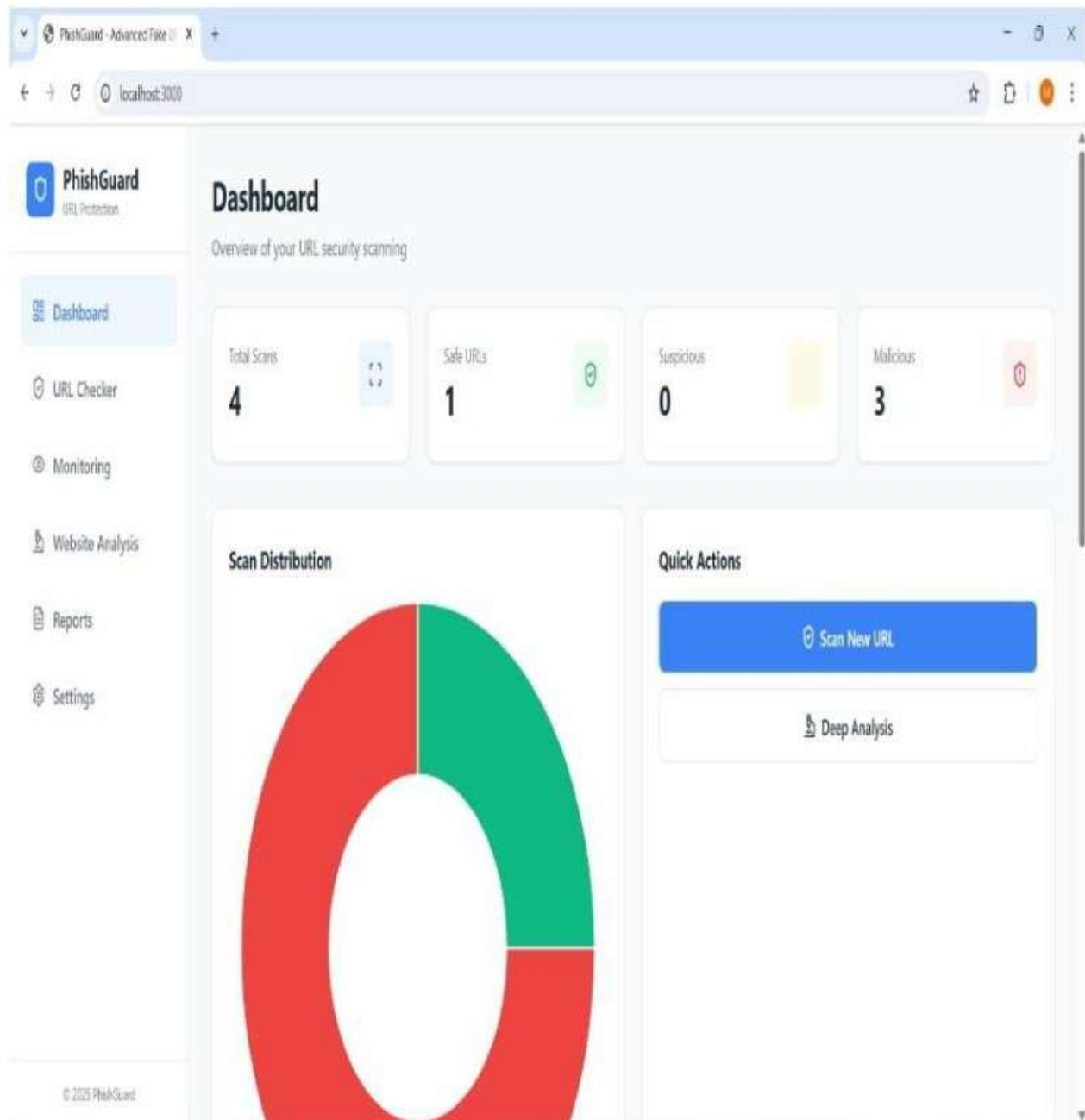
```
<input id="urlInput" placeholder="Enter website URL" />

<button onclick="checkURL()">Check Website</button>

<p id="result"></p>
<script>
async function checkURL() {
    const url = document.getElementById("urlInput").value;
    const response = await fetch("/predict", {
        method: "POST",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify({url})
    });
    const data = await response.json();
    document.getElementById("result").innerText = "Result: " + data.prediction;
}
</script>
```

CHAPTER 6

RESULT



PhishGuard - Advanced Fake U X

localhost:3000

PhishGuard
URL Protection

Dashboard

URL Checker

Monitoring

Website Analysis

Reports

Settings

Reports

Generate and download security reports

Generate Report

Generate HTML Report

Generate CSV Report

Scan History (4)

MALICIOUS Confidence: 86%

<http://update-your-password-now.xyz/>

High-risk indicators: phishing keywords detected

16/11/2025, 10:10:07

MALICIOUS Confidence: 86%

<http://update-your-password-now.xyz/>

High-risk indicators: phishing keywords detected

16/11/2025, 10:10:07

© 2025 PhishGuard



PhishGuard Security Report

Generated: 16/11/2025, 11:06:08

Summary

Total Scans: 4 | Safe: 1 | Suspicious: 0 | Malicious: 3

Scan Results

URL	Status	Confidence	Timestamp
http://update-your-password-now.xyz/	MALICIOUS	88%	16/11/2025, 10:10:07
http://update-your-password-now.xyz/	MALICIOUS	88%	16/11/2025, 10:10:07
http://update-your-password-now.xyz/	MALICIOUS	88%	16/11/2025, 10:10:07
https://www.microsoft.com/	SAFE	36%	16/11/2025, 10:08:43

CHAPTER 7

CONCLUSION & FUTURESCOPE

7.1 Conclusion

The Fake Website Detection System developed in this project provides an intelligent, automated solution to identify fraudulent, suspicious, and phishing websites using machine learning and feature-based analysis. The system addresses one of the most critical security challenges faced by internet users today—online scams and phishing attacks. By extracting essential features from website URLs and webpage structures, the system classifies websites as Safe, Suspicious, or Fake with high accuracy.

Throughout the project, significant emphasis was placed on dataset preparation, feature engineering, model training, and real-time prediction using a backend API. The implementation involved the use of supervised learning algorithms such as Random Forest, Logistic Regression, and Support Vector Machine, enabling robust and reliable predictions. A web-based interface was also developed, making the system accessible and easy to use for non-technical users.

The project demonstrates how AI and machine learning can effectively support cybersecurity systems and protect users from phishing links, fraudulent e-commerce sites, fake banking pages, and malicious URLs. The model's efficient classification performance and the frontend's simplicity reflect a successful integration of data science, backend engineering, and user-focused design.

7.2 Contributions

The major contributions of this project include:

1. Automated Feature Extraction System

A custom-built feature extractor analyzes URLs and webpage content to derive technical indicators such as domain age, SSL usage, URL length, number of redirects, suspicious HTML tags, and more.

2. Machine Learning–Driven Classification

The project introduces a trained model capable of identifying phishing/fake websites with high efficiency, reducing dependency on manual verification.

3. Real-Time URL Checking

Integration of backend API allows users to get predictions instantly without downloading any

software.

4. Simple and User-Friendly Web Interface

The frontend enables users to input URLs and receive classification results with clear visual indicators.

5. Expandable & Modular Architecture

The system is designed in modular form—dataset module, feature extractor, ML model, backend, and UI—which makes the solution maintainable and extendable for future research.

6. Contribution to Cybersecurity Awareness

The project promotes safe browsing habits and equips users with a tool to detect potentially harmful sites.

7.3 Future Work

Several enhancements can further improve the effectiveness and reach of the Fake Website Detection System:

1. Deep Learning Models

Implementing LSTM, CNN, or transformer-based models for URL sequence analysis may significantly increase detection accuracy.

2. Real-Time Webpage Screenshot Analysis

Adding vision-based classification using CNNs to analyze webpage screenshots (e.g., layout similarities with trusted websites).

3. Browser Extension

Building a Chrome/Firefox extension that automatically warns users while browsing suspicious websites.

4. More Advanced Feature Extraction

Including additional features such as:

- JavaScript behavior analysis
- Network-level features (IP reputation, server response time)
- Blacklist/whitelist integration

5. Continuous Learning

Adding mechanisms to automatically update the dataset with new phishing URLs and retrain the model regularly.

6. Multi-Language Support

Providing user interface support in multiple languages to reach more users globally.

7. Mobile App Version

A lightweight mobile app could help users instantly verify URLs shared via SMS, WhatsApp, or social media.

REFERENCES

Research Papers

1. Aburrous, M., Hossain, M. A., Dahal, K., & Thabtah, F. (2010). Intelligent phishing detection system for e-banking using fuzzy data mining. *Expert Systems with Applications*, 37(12), 7913–7921.
2. Mohammad, R. M., Thabtah, F., & McCluskey, L. (2014). Predicting phishing websites based on self-structuring neural network. *Neural Computing and Applications*, 25(2), 443–458.
3. Basnet, R. B., Mukkamala, S., & Sung, A. H. (2012). Detection of phishing attacks: A machine learning approach. *Studies in Fuzziness and Soft Computing*, 296, 373–383.
4. Jain, A. K., & Gupta, B. B. (2018). Phishing detection: Analysis of visual similarity-based approaches. *Security and Communication Networks*, 2018, 1–20.
5. Marchal, S., Saari, K., Singh, N., & Asokan, N. (2016). Know Your Phishy URLs: Phishing Website Detection via a Multi-level Classification Scheme. *IEEE APWG eCrime Researchers Summit*.
6. Zhang, Y., Hong, J. I., & Cranor, L. F. (2007). CANTINA: A content-based approach to detecting phishing websites. *WWW '07: Proceedings of the 16th International Conference on World Wide Web*.
7. Rao, R. S., & Pais, A. R. (2019). Detection of phishing websites using machine learning algorithms. *International Journal of Information Security*, 18(6), 633–651.